

USER MANUAL

A Multimodal Deep Learning Approach for Predicting Transcription Factor Binding in Drosophila

Reproducible preprocessing, model training, evaluation, and in-silico analysis

Author :- Aman Yadav

Project reference: A Multimodal Deep Learning Approach for Predicting Transcription Factor Binding in Drosophila

Purpose and scope

This manual describes how to reproduce the thesis workflow from raw sequencing accessions to model evaluation. It is aligned with the callable classes and functions in the project scripts. The workflow processes ATAC-seq, ChIP-seq, input-control, and dm6 reference data; constructs model-ready arrays; trains DNA-only, ATAC-plus-Tn5, and multimodal models; and performs optional interpretation and in-silico perturbation analyses.

Important: The preprocessing and machine-learning stages use different YAML files. The Snakefile reads config.yaml. The Python model classes read config_file_ml.yaml.

What is included

File	Purpose
Snakefile	The Actual pipeline with its execution rules .
config.yaml	Control unit for preprocessing pipeline
config_file_ml.yaml	Control unit for CNN models(input paths, chromosome splits, seeds, epochs, batches, callbacks, and loss settings.)
DataLoader3.py	Loads ChIP, ATAC, DNA, chromosome, and Tn5-derived arrays into aligned in-memory structures.
Visualizer3.py	Plots ATAC/ChIP relationships, active windows, chromosome distributions, correlations, and genome-wide summaries.
DNA_model3.py	Train the DNA-only prediction model and its evaluation functions.
ATAC_model3.py	Train the ATAC-plus-Tn5 prediction model and its evaluation functions.
ATAC_DNA_model3.py	Train the multimodal DNA-plus-chromatin model and repeated-seed execution.

File	Purpose
Insito_mutation3.py	Performs shuffled-pairing controls, permutation testing, DNA mutation scans, and chromatin perturbations.
environment.yml	Conda software environment and package versions.
script/	Contains helper programs called by the Snakefile, including signal binning and FASTA one-hot encoding.

Computational requirements

Resource use depends on the number of accessions, genomic-window size, sequence length, and neural-network batch size. The following values are practical guidance rather than fixed requirements. (requirement were only considered on 10 chip data sets)

Resource	Minimum for testing	Recommended full run
Operating system	Linux	Linux/HPC cluster
CPU	8 cores	16-32 cores for preprocessing
RAM	32 GB	64 GB or more
GPU	Optional for preprocessing	CUDA-capable GPU; 24-48 GB VRAM is useful for large batches (A5000 was used in the current thesis)
Storage	Depends on selected SRRs	Plan for raw FASTQ, BAM, coverage, peak, and NumPy outputs

Cluster use: Do not run the full workflow or model training on a login node. Request a compute node according to local cluster policy.

Do not use personal pc until it meets the requirements

Eight-step reproduction workflow

Step 1 – Downlaod and verify the project files

Place the Snakefile, both YAML files, environment.yml, metadata CSV, six core Python modules, and the script/ helper directory in the project directory.

(Recommendation)

Make sure their path remains same for smooth execution or the user must change its path and may be problematic first time

Step 2 - Create the Conda environment

Linux code for making Conda environment and downloading tools from environment.yml

```
conda env create -n Testing -f environment.yml
conda activate Testing
python --version
snakemake --version
conda env update -n Testing -f environment.yml --prune
```

Snakemake executor plug-ins do not install the main Snakemake command. If `conda list snakemake` shows only plug-ins and `snakemake --version` fails, install the main package:

!! Snakemake requires separate download

```
conda install -n Testing -c conda-forge -c bioconda snakemake
```

TensorFlow GPU support also require a separate installation after the Conda environment has been created. Confirm that the NVIDIA driver is available, install TensorFlow into the active Testing environment, and verify that TensorFlow detects the allocated GPU (do this only if the main goal is model training and interpretation :

(!! TensorFlow also has separate import for GPU compatibility)

Network access: Problems can arise if the HPC nodes are without internet connection . Makes sure to download it from master node or a node with internet access

Reproducibility: Keep environment.yml under version control. For an exact platform-specific snapshot, also export `conda list --explicit > environment-explicit.txt`.

Step 3 - Configure metadata and preprocessing

The metadata CSV referenced by config.yaml must contain the columns SRR, TYPE, and INPUT_SRR. TYPE identifies ATAC, CHIP, or INPUT datasets. Each ChIP row must point to its matching input-control accession through INPUT_SRR.

The used srr_list.csv file is also provided in Authors git repository. That too must be kept in the same directory

At minimum, config.yaml must provide data_dir, results_dir, threads, srr_csv, windows.size, windows.stride, and the reference download settings used by the Snakefile. Keep chromosome naming consistent with dm6 (for example, chr2L rather than 2L).

Legacy filename: The workflow writes dm6_8kb_windows.bed, but the actual window size and stride are taken from config.yaml. Do not infer the active window size from that historical filename.

Step 4 - Dry-run and execute Snakemake

Always inspect the dependency graph before starting the full workflow:

```
conda activate Testing
snakemake -n -p
snakemake --cores 32
```

On a Slurm cluster, request resources first. The following is only an example and must be adapted to the available partition and cluster policy:

Snakemake reruns only targets that are missing or older than their dependencies. For example, removing results/dna_onehot.npy and restarting Snakemake recreates that target and any downstream output that depends on it.

Step 5 - Validate preprocessing outputs

Before loading data into Python, confirm that the expected model inputs exist and are non-empty.

Output	Expected location
Background-corrected ChIP arrays	results/perbase_chip /{SRR}_perbase.npy
ATAC arrays	results/perbase_atac/{SRR}_perbase.npy
DNA one-hot array	results/dna_onehot.npy
Tn5 fragment BED files	results/ATAC_fragments/{SRR}_fragments.bed
Genomic windows	data/dm6_8kb_windows.bed

The number and order of genomic windows must agree across ChIP, ATAC, DNA, chromosome labels, and derived Tn5 features. Do not continue when the first dimension differs.

Step 6 - Configure machine learning

!! If not updated

Update config_file_ml.yaml so that chip_dir → results/perbase_chip,

atac_dir points → results/perbase_atac,

dna_path → dna_onehot.npy,

chrms_path → window BED file, and

TN5_cutsite points → results/ATAC_fragments.

Set training_chrms, validation_chrms, and test_chrms to disjoint chromosome lists. Every listed chromosome must occur in the loader's chromosome array. The YAML file also controls model_seeds, epochs, batch sizes, callbacks, and the weighted-loss settings for each model.

!! Use the same chromosome split and target ChIP tracks for the DNA-only, ATAC-plus-Tn5, and multimodal models. Change batch size only when required by memory.!!

Step 7 - Load data and run the three models

Import the six project modules in a notebook or Python script:

```
from DataLoader3 import GenomicDataLoader
from Visualizer3 import Visualizer
from DNA_model3 import DNAPredictionModel, run_dna_seed_experiments
from ATAC_model3 import ATACPredictionModel, run_atac_seed_experiments
from ATAC_DNA_model3 import MultimodalGenomicModel, multimodal_seed_execution
```

```
from Insito_mutation3 import GenomicPermuter
```

Run the loading methods in DataLoader3.py until the loader contains concatenated_data, window_chromosomes, dna_tracks, atac_tracks, and names. The model constructors read these attributes directly through from_loader(). The aligned Tn5 feature array must also be available as TN5_fragments.

The execution.ipynb has already prebuilt code by the author and does not require reprogramming, if the task is only result reproducing

These routines use the seed lists in config_file_ml.yaml and report loss, MAE, pooled and track-wise Pearson correlation, Wasserstein distance, Jensen-Shannon divergence, and completed epochs. Results are written under results/seed_runs unless a different output directory is supplied.

Step 8 - Visualization and in-silico analysis

The visualizer is optional for training but important for checking normalization, track naming, active windows, chromosome coverage, ATAC-ChIP correlations, and genome-wide signal behavior.

The Execution.ipynb has its own cell for visualization, just run the cell with specific configuration and limits (a lot of chip can be very heavy process in plots)

The correct-versus-shuffled comparison tests whether predictions depend on the correct genomic pairing. The permutation test estimates an empirical null distribution and p-value. DNA_mutation_test measures prediction sensitivity to single-base substitutions, while ATAC_mutation_test scales chromatin features around a selected signal center. These are sensitivity analyses and must not be described as causal evidence of transcription-factor binding.

Evaluation and interpretation

Metric	Interpretation
Weighted test loss	Training objective on the held-out chromosome; lower is better, but values depend on loss weights.
MAE	Average absolute signal error; lower is better.
Pearson correlation	Agreement in signal variation; higher is better. Report pooled and track-wise values.
Wasserstein distance	Spatial displacement between true and predicted signal distributions in base pairs; lower is better.
Jensen-Shannon divergence	Difference between normalized true and predicted profile shapes; lower is better.

No single metric is sufficient. Compare all three architectures across the same seeds and chromosome split, report mean plus sample standard deviation, and inspect track-level results. Sparse tracks can show low MAE even when peak localization is poor, while elevated background can distort pooled correlation.

Troubleshooting (faced by author)

Problem	Likely cause and correction
snakemake: command not found	Only executor plug-ins are installed, or the wrong Conda environment is active. Activate Testing and install the main snakemake package.
pip reports Network is unreachable	The selected compute node has no outbound internet connection. Install TensorFlow from an internet-enabled node in the same Testing environment, or use the cluster package mirror or offline packages.
TensorFlow detects no GPU	Check nvidia-smi, confirm that a GPU was allocated by the scheduler, and install tensorflow[and-cuda] separately in the active Testing environment.
Snakefile not found	Run from the repository directory or provide -s with the exact filename. Quote filenames containing spaces.
SyntaxError or IndentationError during import	The reported Python module contains an incomplete line, stray character, or indentation mismatch. Run py_compile on all six modules.
from_loader() reports missing attributes	The required DataLoader3.py loading steps were not completed, or a script was renamed internally without updating from_loader().
TN5_frags is not defined	Load or construct the aligned Tn5 feature array before calling the ATAC or multimodal split method. Do not rename full_fragments in only one cell.
Empty chromosome split	Chromosome names in config_file_ml.yaml do not match window_chromosomes, or a split contains no windows.
Out-of-memory error	Reduce the relevant batch size in config_file_ml.yaml, request more RAM/VRAM, or process fewer samples for a test run.

Reproducibility checklist

- Record the repository commit or archive version used for the analysis.
- Keep environment.yml, config.yaml, config_file_ml.yaml, and the metadata CSV with the analysis.
- Record the dm6 reference and annotation source used by the Snakefile.
- Preserve chromosome splits, seed lists, epochs, batch sizes, and loss settings.
- Retain Snakemake logs, model checkpoints, per-seed result files, and failed-run messages.
- Report software versions with conda list and GPU details with nvidia-smi when applicable.
- Do not compare models trained on different targets, preprocessing versions, or chromosome splits as if they were equivalent.

Command reference

```
# Create or update the environment
conda env create -n Testing -f environment.yml
conda env update -n Testing -f environment.yml --prune
conda activate Testing
```

```
# Install TensorFlow GPU support separately when it is not in the environment
```

```
python -m pip install "tensorflow[and-cuda]"
python -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"
```

```
# Verify core commands and scripts
```

```
snakemake --version
```

```
python -m py_compile DataLoader3.py Visualizer3.py DNA_model3.py \
  ATAC_model3.py ATAC_DNA_model3.py Insito_mutation3.py
```

```
# Preprocessing
```

```
snakemake -n -p
```

```
snakemake --cores 32
```

```
# Record versions
```

```
conda env export --from-history > environment-history.yml
```

```
conda list --explicit > environment-explicit.txt
```

```
conda list > package-versions.txt
```